AI INFRA RESEARCH SERIES

Kimi K3

读懂 2.8T MoE、混合注意力与百万上下文训练系统

分析对象

Kimi K3: Open Frontier Intelligence

作者与机构

Kimi Team；Moonshot AI（月之暗面）。

版本范围

arXiv:2607.24653v2；首版 2026-07-27，本文依据 2026-08-07 修订版，重点分析架构与 AI Infra。

核心判断

K3 的价值不在一个孤立新模块，而在同时扩展序列、深度、专家容量和长轨迹 RL，并让数值参数化、kernel、通信与状态管理相互配合。模型能力结果与单项系统收益必须分开归因。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

阅读导航

1 如何阅读这份大模型技术报告 2

2 模型结构：三条信息流，而不是三个同义词 2

3 KDA：递归公式怎样影响 GPU 实现 3

4 Gated MLA：少存 KV 与全局寻址的折中 4

5 Stable LatentMoE：稳定性与负载均衡 4

6 训练系统：把专家、状态和流水线一起调度 5

7 长轨迹 RL：真正昂贵的是状态长时间存活 6

8 如何评价论文的实验与“2.5 倍效率” 6

9 最值得继续研究的三个问题 7

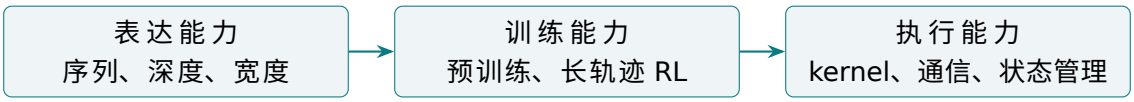
10 最终判断与原始资料 7

文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

1 如何阅读这份大模型技术报告

Kimi K3 是原生多模态 MoE 模型报告，作者给出的规模为 **2.8T 总参数、104B 激活参数、896 个路由专家、每 token 选择 16 个路由专家，以及最多 1M token 上下文**。[S1，§1–2] “激活参数” 描述单 token 涉及的计算，不代表未激活的专家权重可以从训练系统中消失。

整篇可以分成三层。第一层是模型表达：KDA 与 Gated MLA 负责序列交互，Attention Residuals 负责跨深度信息流，Stable LatentMoE 负责稀疏的宽度扩展。第二层是训练：数据与优化器、长期轨迹 RL、多教师蒸馏。第三层是系统：KDA kernel 与 CP、MoonEP、激活生命周期、外部缓存、可恢复执行环境和服务调度。[S1]



本文不会把作者给出的 “整体 scaling efficiency 约 2.5 倍” 改写为 “推理速度提升 2.5 倍”。它描述相对 K2 的架构、训练和数据组合效率，不是只换一个 kernel 的等条件消融。[S1，§1]

2 模型结构：三条信息流，而不是三个同义词

2.1 序列维度：三层 KDA 配一层全局注意力

K3 重复使用 **3 个 KDA 层 + 1 个 Gated MLA 层**，每个注意力子层后都接 Stable LatentMoE；主干末尾再放一个全局 MLA 层。[S1，Fig.2、§2.1]

KDA 以矩阵状态承载历史，每个 decode step 不必读取所有过去的 KV。MLA 保留对全部上下文的内容寻址能力，但以 latent KV 降低存储。混合结构承认了一个现实：固定状态擅长高效积累信息，但复杂检索仍需要部分显式历史访问。

因此，“大多数层用线性注意力” 不等于整个模型的所有计算都线性，也不意味着模型完全没有随上下文增长的 KV Cache。MLA 层仍然承担这部分成本。

2.2 深度维度：Attention Residuals

普通残差把历史层输出不断相加，下一层接收的是一份累计状态。AttnRes 则让当前层有选择地读取较早层的输出。这里 attention 的对象是**层／块的表示**，不是更多历史 token。[S1，§2.2]

令候选层输出为 v_i，使用层特定的可学习 pseudo-query w_l：

$$a_{i\to l}=\frac{\exp(w_l^T\,\mathrm{RMSNorm}(v_i))}{\sum_j \exp(w_l^T\,\mathrm{RMSNorm}(v_j))},\qquad h_l=\sum_i a_{i\to l}v_i.$$

RMSNorm 避免大幅值层仅靠数值大小支配选择。完整形式需要保留很多层的表示，因此作者实际采用 Block AttnRes：块内求和，块间选择性混合，降低跨层存活张量与 pipeline 传输成本。[S1，式 8–10]

从系统角度看，这不是免费加一层 attention。需要决定哪些中间结果保留、哪些跨 stage 传递、在线 softmax 如何合并块结果。表达能力与激活生命周期在这里直接相连。

2.3 宽度维度：Stable LatentMoE

路由专家在较窄的 latent 维度工作，shared experts 保留全宽路径。其结构可写成

$$z = W_{\downarrow} x, \quad u = \sum_{e \in \mathcal{T}_k(x)} p_e E_e(z),$$

$$y = \sum_s E_s^{\text{shared}}(x) + W_{\uparrow}\,\text{RMSNorm}(u).$$

这是对报告结构的整理；具体归一化参数由模型实现决定。减少 routed 路径宽度，可以减小多专家激活的通信与权重读取压力，使扩大专家池更可行。[S1，§2.3]

3 KDA：递归公式怎样影响 GPU 实现

3.1 通道级遗忘与纠错式写入

设 state $S_t \in \mathbb{R}^{d_k \times d_v}$，则 KDA 的核心为

$$S_t = (I - \beta_t k_t k_t^T) D_t S_{t-1} + \beta_t k_t v_t^T, \qquad D_t = \text{Diag}(\alpha_t),$$

$$o_t = S_t^T q_t.$$

先按 key 通道衰减旧状态，再用当前 key 检索旧值，最后把预测误差写回。β 控制写入强度，向量 α 控制不同通道的保留程度。衰减与 rank-one 更新的顺序不能任意交换，因为一般有 $Dkk^T \neq kk^T D$。[S1，式 1；S2，§2]

K3 继承 Kimi Linear 的 chunkwise 实现：chunk 内并行、chunk 间传递状态。它将 token 级递归转为多个矩阵乘与三角变换，使 GPU 不必逐 token 启动小算子。

3.2 最关键的 K3 改动：给 log-decay 加下界

原 Kimi Linear 使用

$$g = -e^A \,\text{softplus}(z), \quad \alpha = e^g,$$

其中 g 可以非常负。chunk 计算涉及累积衰减的倒数；当衰减很小时，倒数可能溢出，迫使实现使用额外的数值处理和位置对计算。[S1，§2.1.1；S2]

K3 改为

$$g = g_{\min}\,\text{sigmoid}(e^A z), \qquad g_{\min} = -5.$$

因此单步 $\alpha > e^{-5}$，16-token tile 的累计 log-decay 大于 -80，对应倒数小于 e^{80}，仍处于 BF16

的指数动态范围内。报告据此让对角与非对角 tile 都能采用密集 Tensor Core 矩阵乘，替代原先对角 tile 的逐位置对路径。[S1，Fig.3、式 5]

这不是把同一数学问题换成更快实现，而是先修改可学习门的取值空间，再获得更规则的计算。限制遗忘速度也改变了模型的归纳偏置，因此要同时检查长程保留、状态重置需求和训练质量。

指数范围安全不代表数值误差为零。BF16 尾数精度、消减误差、三角变换中的误差传播仍然需要验证。不能将 “不溢出” 推导成 “与逐 token FP32 递归逐位相同”。

3.3 full-rank 改的是输出 gate

报告还将 KDA 的输出 gate 从低秩形式改为输入相关的全秩投影：

$$y_t = W_o[\text{sigmoid}(W_g x_t) \odot \text{RMSNorm}(o_t)].$$

这里应准确说 “输出门改为全秩”，不要笼统写成 “所有 KDA gate 都改为全秩”。遗忘门、写入门与输出门承担不同作用。[S1，式 6]

4 Gated MLA：少存 KV 与全局寻址的折中

MLA 通过低维 latent 保存每个 token 的 KV 信息，而不是缓存每个 head 的完整 K 和 V。K3 的全局 MLA 使用 NoPE，不对 Q/K 加显式位置编码；它依靠穿插的 KDA 层提供顺序敏感和近因相关的信息处理。MLA 输出也加了输入相关通道门。[S1，§2.1.2]

这种设计不是 “模型不需要位置”，而是把位置与顺序信息的职责放到混合结构里。全局层专注内容交互，递归层提供带方向的更新。

报告还有一个很贴近算子优化的细节：为了减轻注意力输出的有偏舍入，训练保留 FP32 attention output；更大的片上输出 tile 会挤压共享内存，因此需要重排输出与 KV staging buffer 的生命周期，维持足够深的流水线。[S1，§2.1.2]

这说明更高精度与更快 kernel 并不总是对立。关键是能否重新安排片上存储，而不是只问 “能否把某个张量 cast 成 BF16”。

5 Stable LatentMoE：稳定性与负载均衡

5.1 为什么需要 SiTU-GLU

latent down-projection、专家 GLU 与 up-projection 连在一起，会形成较长的线性变换链。极大模型与高度稀疏路由下，内部激活放大可能更难控制。作者使用 routed 汇总后的归一化，并将 GLU 两个分支平滑限幅。[S1，§2.3.1–2.3.2]

设 gate 分支输入 a、up 分支输入 b，其示意形式为

$$\phi(a,b) = [\beta_1 \tanh(a/\beta_1)\sigma(a)]\,[\beta_2 \tanh(b/\beta_2)]\,.$$

绝对值上界为 $\beta_1\beta_2$。靠近原点时近似 SwiGLU，远处平滑饱和。与硬 clamp 相比，它仍有连续的梯

度变化，但饱和区也会降低敏感性。Fig.4 的示例采用 $\beta_1 = 4, \beta_2 = 25$，不能据图示把所有实现参数都视为自由等价选择。[S1，Fig.4、附录 B]

5.2 Quantile Balancing 不是 MoonEP

Quantile Balancing（QB）调整**逻辑专家的选择偏置**。MoonEP 保持给定的选择，调整**这些专家在哪个 rank 执行**。前者影响训练路由分布，后者解决某个 microbatch 的物理负载；两者不能互换名称。

QB 对 token 产生 sigmoid 原始分数 s_{ij}，top-K 选择使用 $s_{ij} + b_j$，但混合概率由所选原始分数归一化得到。训练期每次获得 top-$(k+1)$ 的 cutoff a_i，再从每个专家的 margin 分布求目标分位数：

$$\hat{b}_j^{t+1} = -\,\text{quantile}_{1-k/n}(s_{:,j} - a^t),$$

随后去掉所有专家偏置的共同均值。直观上，它直接寻找能达到目标接收量的门槛，而不是仅按负载误差的正负做固定步长调整。[S1，§2.3.3]

实际实现用全局直方图估计分位数，避免 all-gather 所有 token×expert 分数。各 rank 的计数可相加，因此 “先聚合直方图再取分位数” 保留了 pooled global batch 的含义；平均每个 rank 的分位数一般不等价。[S1，附录 D]

偏置只在下一步生效，推理时冻结。固定 cutoff 的理想推导不代表当前实现每个动态 microbatch 都严格等负载：离散 ties、直方图精度、下一步数据分布变化都会影响结果。这正是物理侧仍需要 MoonEP 的原因。

6 训练系统：把专家、状态和流水线一起调度

6.1 MoonEP 与专家计算的关系

MoonEP 从当前路由结果构造复制与执行计划，给空闲 rank 临时放热门专家副本，让各 rank 处理同样数量的真实路由 token。副本权重在计算前预取，训练梯度回收到原专家。[S1，§5.2.1、附录 E；S3]

但 token 数相同不等于 kernel 时间相同。一个 rank 可能处理少量大专家块，另一个处理很多小块；padding、tile 数和数据类型也会改变工作量。K3 因而还设计 workload-aware 的 grouped GEMM 调度，对计算块进行代价估计与安排。[S1，§5.2]

通信库消除的是 rank 级 token 偏斜；grouped GEMM 调度继续处理 rank 内的不规则性。

6.2 KDA Context Parallelism

full attention 的 CP 处理 Q/K/V 的序列切分；KDA CP 的关键则是各段之间的状态依赖。把长序列分给多个 rank 后，不能假设每段从零状态开始还能与完整序列等价。

从递归式看，一段历史对输入 state 的作用是一个仿射变换：

$$S_{\text{out}} = A_{\text{segment}} S_{\text{in}} + B_{\text{segment}}.$$

这是理解状态衔接的数学视角，不表示代码必然显式存储巨大稠密 A。实际设计需要用结构化表示与通信协议保持段间依赖，并比较状态传输成本与段内 GEMM 收益。[S1，§5.1]

6.3 数量级 sanity check

2.8T 参数仅以 BF16 保存一次就约需 $2.8 \times 10^{12} \times 2 = 5.6$ TB，尚未计梯度、优化器、激活、通信池或副本。这个算式是容量估算，不是作者实测显存。

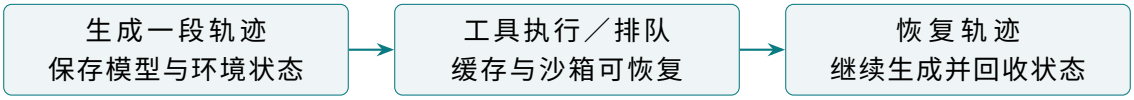
它说明 104B 激活参数不能被当作部署全部权重的内存账单。低精度、分片、offload 和 EP 不是可有可无的配件，而是模型可训练、可部署的基础条件。

7 长轨迹 RL：真正昂贵的是状态长时间存活

K3 的 post-training 涉及不同任务域和 reasoning effort：长程编程、工具使用、通用推理、多模态与持续助手任务，再通过多教师 on-policy 蒸馏整合能力。[S1，§4]

对系统而言，长 agent 轨迹除了 token 还带来工具等待、沙箱、文件系统、历史 KV、递归状态和随机状态。只优化 rollout 的一个 forward，不能解决长时间挂起后再恢复的问题。

报告采用 partial rollout、外部 KV 保留、可恢复 microVM，以及自适应限流，在训练与 rollout 共置条件下管理模型和环境的长期状态。[S1，§5.3]



这里最容易忽视的是版本一致性：继续同一条轨迹时，模型参数版本、已缓存 KV/state、路由与随机状态必须有明确语义。报告给出系统设计，但不应把“可恢复”自动理解为任意策略更新后都能无条件复用旧状态。

7.1 混合模型的 prefix cache 不只是一张 KV 表

MLA 历史可以按页保存，KDA 则有递归 state 和短卷积历史。共享前缀命中时，需要恢复同一前缀边界的组合状态；分叉后两个请求必须隔离后续写入。

因此，前缀缓存键、checkpoint 边界、copy-on-write、驱逐和恢复粒度都需要理解模型结构。KDA 固定大小的 state 降低了单步 decode 历史成本，却让“任意位置回退”需要额外 checkpoint 或重算，这不是免费的历史随机访问。[S1，§5.1、§5.4；此段包含解释性推导]

8 如何评价论文的实验与“2.5 倍效率”

这是综合技术报告，不是只改变一个因素的实验。应分开看三类证据：相近规模的架构／训练对比、特定算子的性能测量，以及最终大模型的能力评测。后者不能独立证明某个 kernel 或某种路由策略贡献了多少。

论文中的说法	正确理解	不应推出的结论
2.8T / 104B	总参数与单 token 激活规模不同	权重只占 104B 对应内存
1M 上下文	在约定窗口内支持长序列任务	任意位置的信息均可无损记住
约 2.5× scaling efficiency	架构、训练与数据组合的效率	任意推理请求快 2.5 倍
MoonEP 完美均衡	给定条件下平衡真实 token 数	所有 kernel 时间完全相等
最终评测领先若干基线	报告所用版本与评测设置的结果	在所有成本与工具预算下都领先

能力评测包含工具环境、思考预算、采样次数及 harness；对比时必须检查这些因素。随着闭源模型和基准版本更新，不宜把 8 月报告的排名直接写成 10 月底的实时排名。本报告因此不以排行榜名次作为技术优劣的主要依据。[S1，§6]

论文对预训练数据、全规模超参数和生产环境的披露也不意味着外部团队能以同样成本完整复现。更现实的验证是挑选可隔离的机制，做小规模等条件实验。

9 最值得继续研究的三个问题

第一，数值范围能否指导 kernel 架构? 固定 KDA 输入，比较无界与下界 log-decay 的 recurrent FP32、chunk BF16 输出和梯度，记录溢出、误差与 Tensor Core 路径占比。研究目标应是质量—数值稳定—吞吐的共同曲线，而不只是最快 kernel。

第二，混合 state/KV 缓存的成本模型。同一前缀不同分叉数、不同回退长度下，比较保存多少 recurrent checkpoints、何时重算、哪些 MLA 页保留。用每请求持久内存、恢复延迟、命中后有效速度和错误率共同评价。

第三，逻辑均衡与物理均衡的组合。固定 router 输出轨迹，分别启用 QB 风格的逻辑偏置、MoonEP 执行迁移及 grouped GEMM 调度。每项优化改变的变量不同，只有拆开测，才能判断收益来源。

这三个方向都与 AI Infra 紧密相关，但本报告没有把它们宣称为已验证的新论文贡献。

10 最终判断与原始资料

K3 最值得学习的是：它并不把模型设计、编译优化、通信库和 RL 执行环境视为四个互不相干的团队任务。一次门函数参数化的改变，可以让 kernel 更规则；一次缓存语义的改变，可以让长 agent 轨迹更容易调度。

[S1] Kimi K3 v2。 arXiv 2607.24653v2。重点：§2.1–2.3、§5.1–5.4、附录 B–E。本文的架构事实、参数规模和作者自述实验均来自该版本。

[S2] Kimi Linear v2。 arXiv 2510.26692v2。KDA 的原始递归、chunkwise 算法与 3:1 混合架构来源。K3 的有界 log-decay 和全秩输出门是后续改动。

[S3] MoonEP 官方仓库。 固定版本 README。本套第六份报告单独解析其接口、梯度与基准。

本报告未独立训练 K3、执行长上下文 RL 或复跑生产评测。公式整理、显存数量级估算与复现设计为解释性分析，不是额外实测数据。